

WEAK-4

DESIGN ORDERS AND ORDERS DETAILS TABLES

AIM

To design and create the Orders and Order_Details tables in SQL with appropriate primary keys, foreign keys, NOT NULL, and CHECK constraints to maintain data integrity and establish relationships between orders and products.

CREATE TABLES

ORDERS AND ORDER_DETAILS TABLE:

```
SQL> CREATE TABLE Orders (Order_ID int, Customer_ID VARCHAR2(7), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(10));  
Table created.  
  
SQL> CREATE TABLE Order_Details (Order_Detail_ID VARCHAR2(7), Order_ID int, Product_ID VARCHAR2(7), Quantity NUMBER(5), Price NUMBER(10,2)  
)  
Table created.
```

INSERT FOR ORDERS TABLE:

```
SQL> INSERT INTO Orders VALUES (1001, 'C1001', SYSDATE, 67500, 'Pending');  
1 row created.  
  
SQL> INSERT INTO Orders VALUES (1002, 'C1002', SYSDATE, 25000, 'Shipped');  
1 row created.  
  
SQL> INSERT INTO Orders VALUES (1003, 'C1003', SYSDATE, 1500, 'Delivered');  
1 row created.  
  
SQL> INSERT INTO Orders VALUES (1004, 'C1004', SYSDATE, 4200, 'Pending');  
1 row created.  
  
SQL> INSERT INTO Orders VALUES (1005, 'C1005', SYSDATE, 8000, 'Delivered');  
1 row created.
```

INSERT FOR ORDER_DETAILS TABLE:

```
SQL> INSERT INTO Order_Details VALUES ('OD001',1001,'P1001',1,67500);
1 row created.

SQL> INSERT INTO Order_Details VALUES ('OD002',1002,'P1002',1,25000);
1 row created.

SQL> INSERT INTO Order_Details VALUES ('OD003',1003,'P1003',2,750);
1 row created.

SQL> INSERT INTO Order_Details VALUES ('OD004',1004,'P1004',2,2100);
1 row created.

SQL> INSERT INTO Order_Details VALUES ('OD005',1005,'P1005',1,8000);
1 row created.
```

SELECT FOR ORDER AND ORDER_DETAILS:

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOME	ORDER_DAT	TOTAL_AMOUNT	ORDER_STAT
1001	C1001	27-SEP-26	67500	Pending
1002	C1002	27-SEP-26	25000	Shipped
1003	C1003	27-SEP-26	1500	Delivered
1004	C1004	27-SEP-26	4200	Pending
1005	C1005	27-SEP-26	8000	Delivered

```
SQL> SELECT * FROM Order_Details;
```

ORDER_D	ORDER_ID	PRODUCT	QUANTITY	PRICE
OD001	1001	P1001	1	67500
OD002	1002	P1002	1	25000
OD003	1003	P1003	2	750
OD004	1004	P1004	2	2100
OD005	1005	P1005	1	8000

PRIMARY KEY FOR TWO TABLES:

Uniquely identifies each record in the table.

```
SQL> ALTER TABLE Orders ADD PRIMARY KEY (Order_ID);  
Table altered.  
  
SQL> ALTER TABLE Order_Details ADD PRIMARY KEY (Order_Detail_ID);  
Table altered.
```

FOREIGN KEY:

Establishes a relationship between two tables.

```
SQL> ALTER TABLE Order_Details ADD CONSTRAINT FK_OrderDetails_Orders FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);  
Table altered.
```

NOT NULL:

Ensures a column cannot contain empty values.

```
SQL> ALTER TABLE Orders MODIFY Customer_ID NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Order_Details MODIFY ORDER_ID NOT NULL;  
Table altered.
```

CHECK:

Ensures that column values satisfy a specified condition.

```
SQL> ALTER TABLE Orders ADD CONSTRAINT CK_Orders_Amount CHECK (Total_Amount > 0);  
Table altered.  
  
SQL> ALTER TABLE Order_Details ADD CONSTRAINT CK_OrderDetails_Price CHECK (Price > 0);  
Table altered.
```

Update:

Used to modify existing order details, such as changing the quantity or price.

```
SQL> UPDATE Order_Details SET Price = 800 WHERE Order_Detail_ID = 'OD003';
1 row updated.

SQL> SELECT * FROM Order_Details;
```

ORDER_D	ORDER_ID	PRODUCT	QUANTITY	PRICE
OD001	1001	P1001	1	67500
OD002	1002	P1002	1	25000
OD003	1003	P1003	2	800
OD004	1004	P1004	2	2100

DELETE:

Used to remove an order or order detail record from the table.

```
SQL> DELETE FROM Order_Details WHERE Order_ID = 1005;
1 row deleted.

SQL> SELECT * FROM Order_Details;
```

ORDER_D	ORDER_ID	PRODUCT	QUANTITY	PRICE
OD001	1001	P1001	1	67500
OD002	1002	P1002	1	25000
OD003	1003	P1003	2	750
OD004	1004	P1004	2	2100

Report 1: Customer Order History

This report displays each customer's order details, including the order ID, date, amount, and status.

It helps track the complete order history of customers.

```
SQL> SELECT C.Customer_Name, O.Order_ID, O.Order_Date, O.Total_Amount, O.Order_Status FROM Customer C JOIN Orders O ON C.Customer_ID = O.Customer_ID;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DATE
Arun	1001	27-SEP-26
67500 Pending		
Priya	1002	27-SEP-26
25000 Shipped		
Kumar	1003	27-SEP-26
1500 Delivered		

CUSTOMER_NAME	ORDER_ID	ORDER_DATE
Divya	1004	27-SEP-26
4200 Pending		
Rahul	1005	27-SEP-26
8000 Delivered		

Report 2: Product-wise Order Report

This report shows how many times each product was ordered and the total quantity sold.

It helps analyze the sales of each product.

```
SQL> SELECT P.Product_Name, COUNT(OD.Order_ID) AS Times_Ordered, SUM(OD.Quantity) AS Total_Quantity_Sold FROM Product P JOIN Order_Details OD ON P.Product_ID = OD.Product_ID GROUP BY P.Product_Name;
```

PRODUCT_NAME	TIMES_ORDERED	TOTAL_QUANTITY_SOLD
Laptops	1	1
Mobile	1	1
TShirt	1	2
Shoes	1	2

Report 3: Customer Purchase Analysis

This report shows the total number of orders and total spending of each customer.

It helps analyze customers' purchasing patterns and average order value.

```
SQL> SELECT C.Customer_Name, COUNT(O.Order_ID) AS Total_Orders, SUM(O.Total_Amount) AS Total_Spending, AVG(O.Total_Amount) AS Average_Order_Value FROM Customer C JOIN Orders O ON C.Customer_ID = O.Customer_ID GROUP BY C.Customer_Name ORDER BY Total_Orders DESC;
```

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
Arun	1	67500	67500
Priya	1	25000	25000
Rahul	1	8000	8000

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
Divya	1	4200	4200
Kumar	1	1500	1500

Result

The **Customer Order History** report successfully displayed customers and their order details.

The **Product-wise Order Report** showed the number of orders and total quantity sold for each product.

The **Customer Purchase Analysis** displayed each customer's total orders, total spending, and average order value.